O que é a Ordenação por Seleção?

- O *Selection Sort* é um algoritmo de ordenação simples: a cada passo, **seleciona o menor elemento** do trecho ainda não ordenado e o coloca em sua posição correta.
- A cada execução parcial, o algoritmo:
  - **percorre** todo o trecho restante à procura do menor valor (custo $\Theta(n)$);
  - **troca** esse valor com o primeiro elemento do trecho restante;
  - faz **apenas uma** chamada recursiva sobre o restante do vetor, agora com **um elemento a menos**.

Diferença importante em relação às buscas

Aqui, o custo de **cada chamada não é constante**: encontrar o menor elemento exige percorrer todo o trecho restante, cujo tamanho diminui a cada passo.

Pseudocódigo da Ordenação por Seleção (Rec

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1 função selecao(v, ini, fim):
2     if ini ≥ fim then
3         return ;                          // condição de parada
4     menor ← ini;
5     for j ← ini + 1 to fim do
6         if v[j] < v[menor] then
7             menor ← j;
8     troca v[ini] e v[menor];
9     selecao(v, ini + 1, fim);
```

Pseudocódigo da Ordenação por Seleção (I

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1 função selecaoIterativa(v, ini, fim):
2     for i ← ini to fim − 1 do
3         menor ← i;
4         for j ← i + 1 to fim do
5             if v[j] < v[menor] then
6                 menor ← j;
7         troca v[i] e v[menor];
```

Mesma ideia da versão recursiva, mas com **dois laços aninhados**: o externo percorre as posições, o interno procura o menor elemento restante.

Exemplo passo a passo

Vamos simular a Ordenação por Seleção no mesmo vetor reduzido usado no Bubble Sort e no Insertion Sort:

$$v = [\,5,\ 2,\ 8,\ -3,\ 9,\ 1\,]$$

Em cada passagem, buscamos o menor elemento do trecho restante:

vermelho — o elemento da posição atual e o **menor encontrado** no trecho restante (antes da troca);

azul — as mesmas duas posições, já **trocadas** (depois).

Quando o menor elemento já está na posição atual, apenas um destaque aparece — não há troca.

[5 , 2, 8, -3 , 9, 1] → [-3 , 2, 8, 5 , 9, 1]

O menor elemento do vetor inteiro é −3: troca com a posição 0.

$$[\ -3,\ 2\ ,\ 8,\ 5,\ 9,\ 1\ ] \rightarrow [\ -3,\ 1\ ,\ 8,\ 5,\ 9,\ 2\ ]$$

O menor do trecho restante $[2, 8, 5, 9, 1]$ é 1: troca com a posição 1.

$$[\ -3,\ 1,\ \boxed{8},\ 5,\ 9,\ \boxed{2}\ ] \rightarrow [\ -3,\ 1,\ \boxed{2},\ 5,\ 9,\ \boxed{8}\ ]$$

O menor do trecho restante $[8, 5, 9, 2]$ é 2: troca com a posição 2.

$$[\ -3,\ 1,\ 2,\ 5\ ,\ 9,\ 8\ ] \rightarrow [\ -3,\ 1,\ 2,\ 5\ ,\ 9,\ 8\ ]$$

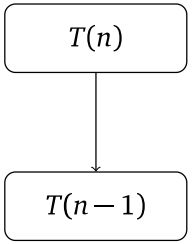
O menor do trecho restante $[5, 9, 8]$ é o próprio 5: nenhuma troca é necessária.

$$[\ -3,\ 1,\ 2,\ 5,\ 9,\ 8\ ] \rightarrow [\ -3,\ 1,\ 2,\ 5,\ 8,\ 9\ ]$$

O menor do trecho restante $[9, 8]$ é 8: troca com a posição 4.

Vetor totalmente ordenado:

$$[\ -3,\ 1,\ 2,\ 5,\ 8,\ 9\ ]$$



- Apenas **uma** chamada recursiva é feita, sobre o vetor **sem o elemento já posicionado**.
- Antes de recursar, o algoritmo **percorre todo o trecho restante** para encontrar o menor elemento: custo $\Theta(n)$.
- Diferente das buscas, esse custo **não é constante** — depende do tamanho da entrada atual.

A relação de recorrência é usada para analisar a **versão recursiva**. A primeira etapa para identificar a classe de complexidade da Ordenação por Seleção é escrever essa relação:

Formulação inicial

$$T(n) = T(n-1) + \Theta(n)$$

A relação de recorrência é usada para analisar a **versão recursiva**. A primeira etapa para identificar a classe de complexidade da Ordenação por Seleção é escrever essa relação:

Formulação inicial

$$T(n) = T(n-1) + \Theta(n)$$

Simplificando:

Forma simplificada

$$T(n) = T(n-1) + n$$

onde $n = fim - ini + 1$ (o tamanho do trecho ainda não ordenado).

$$T(n) = \underbrace{T(n-1)}_{\text{chamada recursiva}} + \underbrace{n}_{\text{busca pelo menor}}$$

$T(n-1)$

Representa a **única chamada recursiva**, feita sobre o restante do vetor, que tem **um elemento a menos** que a entrada atual (o menor já foi posicionado).

$+n$

Representa o custo de **percorrer** o trecho restante procurando o menor elemento.

- Precisa comparar **todos** os elementos do trecho;
- Por isso, custo **linear**: $\Theta(n)$.

$$T(n) = T(n-1) + n$$

Atenção

Assim como na Busca Linear, essa recorrência **não** está no formato exigido pelo Teorema Mestre, pois a entrada não é dividida por um fator b — ela apenas **decresce em 1** a cada chamada.

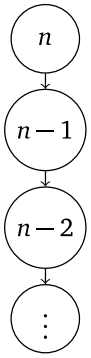
Além disso, aqui o custo fora da chamada recursiva **não é constante**: ele muda a cada nível. Vamos resolver pela **árvore de recursão**.

Vamos ilustrar a árvore de recursão gerada por

$$T(n) = T(n-1) + n$$

Diferenças em relação às árvores anteriores:

- A árvore continua sendo uma **cadeia** (um nó por nível), pois há apenas **1 chamada recursiva** por passo;
- Cada nó filho reduz a entrada do nó pai em **apenas 1** → resume-se em $T(n-1)$;
- Diferente da Busca Linear, o custo de cada nó **não é constante**: ele é proporcional ao tamanho da entrada daquele nó → resume-se no termo $+n$.



1

Nível 0:	custo n
Nível 1:	custo $n-1$
Nível 2:	custo $n-2$
$\vdots$	$\vdots$
Nível h:	custo 1

A árvore tem **um nó por nível**, mas o custo de cada nível **diminui** conforme a árvore cresce.

Calculando a altura da árvore

A árvore para de crescer quando o tamanho da entrada chega a 0 (condição de parada `ini >= fim`):

$$n - h = 0$$

Calculando a altura da árvore

A árvore para de crescer quando o tamanho da entrada chega a 0 (condição de parada `ini >= fim`):

$$n - h = 0$$

Isolando h:

$$h = n$$

Assim como na Busca Linear, a altura da árvore é **proporcional a** n. A diferença agora é que **o custo por nível não é o mesmo em todos os níveis**.

Como o custo de cada nível **muda** a cada nível, não podemos simplesmente multiplicar altura por custo. Precisamos **somar** os custos de todos os níveis:

$$f(n) = n + (n-1) + (n-2) + \cdots + 2 + 1$$

Como o custo de cada nível **muda** a cada nível, não podemos simplesmente multiplicar altura por custo. Precisamos **somar** os custos de todos os níveis:

$$f(n) = n + (n-1) + (n-2) + \cdots + 2 + 1$$

Essa é a soma dos n primeiros números naturais, cujo resultado é conhecido:

Tempo total

$$f(n) = \frac{n \cdot (n+1)}{2} = \frac{n^2 + n}{2}$$

O termo dominante é n^2, logo $f(n) = \Theta(n^2)$.

$$T(n) = \Theta(n^2)$$

- Resolvendo a recorrência $T(n) = T(n-1) + n$ pela árvore de recursão, obtemos um tempo de execução **quadrático**;
- A Ordenação por Seleção tem complexidade $\Theta(n^2)$ em **todos os casos** (melhor, médio e pior) — ela sempre percorre o trecho restante inteiro em busca do menor elemento, mesmo que o vetor já esteja ordenado;
- É mais lenta que o MergeSort ($\Theta(n \log n)$), mas mais simples de implementar e não usa memória extra.

Analisando a versão iterativa

A versão iterativa não é analisada por recorrência, mas o raciocínio é direto: basta contar quantas vezes os laços aninhados executam.

- O laço **externo** executa n vezes (uma para cada posição a preencher);
- Para cada posição i, o laço **interno** percorre os elementos restantes: $n - i$ comparações;
- Somando todas as iterações do laço interno:

Analisando a versão iterativa

A versão iterativa não é analisada por recorrência, mas o raciocínio é direto: basta contar quantas vezes os laços aninhados executam.

- O laço **externo** executa n vezes (uma para cada posição a preencher);
- Para cada posição i, o laço **interno** percorre os elementos restantes: $n - i$ comparações;
- Somando todas as iterações do laço interno:

Tempo total

$$f(n) = n + (n-1) + \cdots + 1 = \frac{n(n+1)}{2} \Rightarrow T(n) = \Theta(n^2)$$

Mesma complexidade de tempo da versão recursiva, mas sem o custo de **espaço** das chamadas na pilha de recursão.

	Recursiva	Iterativa
Tempo (todos os casos)	$\Theta(n^2)$	$\Theta(n^2)$
Espaço extra	$\Theta(n)$ (pilha)	$\Theta(1)$
Análise	recorrência $\rightarrow$ árvore	laços aninhados

- As duas versões fazem o **mesmo número de comparações**, em qualquer caso;
- A diferença está no uso de memória: cada chamada recursiva ocupa espaço na pilha, enquanto a iterativa reutiliza as mesmas variáveis.

- A Ordenação por Seleção, a cada passo, escolhe o **menor elemento** do trecho restante e o posiciona.
- **Versão recursiva**: $T(n) = T(n-1) + n$, resolvida pela árvore de recursão (altura $h = n$, custos $n, n-1, \ldots, 1$).
- **Versão iterativa**: dois laços aninhados, somando $n + (n-1) + \cdots + 1$ comparações.
- Ambas as versões chegam à mesma complexidade final: $T(n) = \Theta(n^2)$, em todos os casos.
- A versão iterativa evita o custo de espaço extra da pilha de recursão.

Dúvidas?

Perguntas? Vamos programar?



O que é o Bubble Sort?

- É um algoritmo de ordenação simples: a cada passagem, **compara elementos adjacentes** e os **troca de posição** quando estão fora de ordem.
- A cada execução parcial, o algoritmo:
  - percorre o trecho ainda não ordenado, comparando cada par $(v[j], v[j+1])$ e trocando-os se necessário: custo $\Theta(n)$;
  - ao final da passagem, o **maior elemento** do trecho já está na sua posição final (foi "borbulhado" até o fim);
  - faz **apenas uma** chamada recursiva sobre o restante do vetor, agora com **um elemento a menos** para ordenar.

Por que "Bubble" Sort?

A cada passagem, o maior elemento ainda não posicionado "sobe" (borbulha) gradualmente até sua posição correta, através de trocas sucessivas com o vizinho.

Pseudocódigo do Bubble Sort (Recursiva)

Universidade de Fortaleza

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1 função bubbleSort(v, ini, fim):
2     if fim ≤ ini then
3         return ;                          // condição de parada
4     for j ← ini to fim − 1 do
5         if v[j] > v[j + 1] then
6             troca v[j] e v[j + 1];
7     bubbleSort(v, ini, fim − 1) ;         // maior já está em fim
```

Pseudocódigo do Bubble Sort (Iterativa)

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1 função bubbleSortIterativo(v, ini, fim):
2     for i ← fim to ini + 1 do
3         for j ← ini to i − 1 do
4             if v[j] > v[j + 1] then
5                 troca v[j] e v[j + 1];
```

O laço externo percorre i de forma **decrescente**, de *fim* até $ini + 1$. Mesma ideia da versão recursiva, mas com **dois laços aninhados**: o externo controla até onde a passagem deve ir, o interno faz as comparações e trocas adjacentes.

Exemplo passo a passo

Para tornar o funcionamento do algoritmo bem visível, vamos simular o Bubble Sort em um vetor pequeno:

$$v = [\,5,\ 2,\ 8,\ -3,\ 9,\ 1\,]$$

Em cada passagem, comparamos **pares adjacentes**:

vermelho — os dois elementos foram comparados e **não trocaram** de posição;

azul — os dois elementos foram comparados e **trocaram** de posição.

Ao final de cada passagem, o maior elemento restante já está na sua posição final (mais à direita).

[2 , 5 , 8, –3, 9, 1]

[2, 5 , 8 , –3, 9, 1]

[2, 5, -3 , 8 , 9, 1]

[2, 5, –3, 8 , 9 , 1]

[2, 5, –3, 8, 1 , 9]

O 9 já está em sua posição final.

[2 , 5 , −3, 8, 1, 9]

[2, -3 , 5 , 8, 1, 9]

[2, −3, 5 , 8 , 1, 9]

[2, −3, 5, 1 , 8 , 9]

O 8 já está em sua posição final.

[-3 , 2 , 5, 1, 8, 9]

[–3, 2 , 5 , 1, 8, 9]

[–3, 2, 1 , 5 , 8, 9]

O 5 já está em sua posição final.

[-3 , 2 , 1, 5, 8, 9]

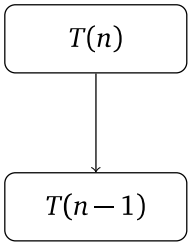
[−3, 1 , 2 , 5, 8, 9]

O 2 já está em sua posição final.

[-3 , 1 , 2, 5, 8, 9]

Vetor totalmente ordenado:

[−3, 1, 2, 5, 8, 9]



- Apenas **uma** chamada recursiva é feita, sobre o vetor **sem o maior elemento**, que já foi posicionado ao final.
- Antes de recursar, o algoritmo faz **uma passagem completa** pelo trecho atual, comparando pares adjacentes: custo $\Theta(n)$.
- Assim como na Ordenação por Seleção, esse custo **não é constante** — depende do tamanho da entrada atual.

A relação de recorrência é usada para analisar a **versão recursiva**. A primeira etapa para identificar a classe de complexidade do Bubble Sort é escrever essa relação:

Formulação inicial

$$T(n) = T(n-1) + \Theta(n)$$

A relação de recorrência é usada para analisar a **versão recursiva**. A primeira etapa para identificar a classe de complexidade do Bubble Sort é escrever essa relação:

Formulação inicial

$$T(n) = T(n-1) + \Theta(n)$$

Simplificando:

Forma simplificada

$$T(n) = T(n-1) + n$$

onde $n = fim - ini + 1$ (o tamanho do trecho ainda não ordenado).

Interpretando cada termo

$$T(n) = \underbrace{T(n-1)}_{\text{chamada recursiva}} + \underbrace{n}_{\text{passagem de comparações}}$$

$T(n-1)$

Representa a **única chamada recursiva**, feita sobre o restante do vetor, que tem **um elemento a menos** (o maior já foi posicionado ao final).

$+n$

Representa o custo de **percorrer** o trecho atual comparando pares adjacentes e trocando-os quando necessário.

- Precisa comparar **todos** os pares vizinhos do trecho;
- Por isso, custo **linear**: $\Theta(n)$.

$$T(n) = T(n-1) + n$$

Atenção

Essa é **exatamente a mesma recorrência** da Ordenação por Seleção: não se encaixa no Teorema Mestre (decresce em 1, não divide por b) e o custo fora da chamada recursiva **muda** a cada nível.

Vamos resolvê-la, mais uma vez, pela **árvore de recursão**.

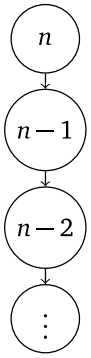
Vamos ilustrar a árvore de recursão gerada por

$$T(n) = T(n-1) + n$$

Características dessa árvore:

- É uma **cadeia** (um nó por nível), pois há apenas **1 chamada recursiva** por passo;
- Cada nó filho reduz a entrada do nó pai em **apenas 1** $\rightarrow$ resume-se em $T(n-1)$;
- O custo de cada nó é proporcional ao tamanho da sua entrada, e por isso **diminui** a cada nível $\rightarrow$ resume-se no termo $+n$.

Árvore de Recursão do Bubble Sort



1

Nível 0:	custo n
Nível 1:	custo $n-1$
Nível 2:	custo $n-2$
$\vdots$	$\vdots$
Nível h:	custo 1

A cada passagem, o trecho a percorrer é **um elemento menor** — a mesma estrutura vista na Ordenação por Seleção.

A árvore para de crescer quando o tamanho do trecho chega a 0 (condição de parada `fim <= ini`):

$$n - h = 0$$

A árvore para de crescer quando o tamanho do trecho chega a 0 (condição de parada `fim <= ini`):

$$n - h = 0$$

Isolando h:

$$h = n$$

A altura da árvore é **proporcional a** n, e o custo por nível **não é o mesmo** em todos os níveis.

Como o custo de cada nível **muda** a cada nível, precisamos **somar** os custos de todos os níveis:

$$f(n) = n + (n-1) + (n-2) + \cdots + 2 + 1$$

Como o custo de cada nível **muda** a cada nível, precisamos **somar** os custos de todos os níveis:

$$f(n) = n + (n-1) + (n-2) + \cdots + 2 + 1$$

Essa é a soma dos n primeiros números naturais:

Tempo total

$$f(n) = \frac{n \cdot (n+1)}{2} = \frac{n^2 + n}{2}$$

O termo dominante é n^2, logo $f(n) = \Theta(n^2)$.

$$T(n) = \Theta(n^2)$$

- Resolvendo a recorrência $T(n) = T(n-1) + n$ pela árvore de recursão, obtemos um tempo de execução **quadrático**;
- No **pior caso** (vetor em ordem decrescente) e no **caso médio**, o Bubble Sort tem complexidade $\Theta(n^2)$;
- No **melhor caso** (vetor já ordenado), uma versão otimizada que **para antecipadamente** ao não detectar trocas em uma passagem alcança $\Theta(n)$ — mas o pseudocódigo apresentado aqui não inclui essa otimização.

Analisando a versão iterativa

A versão iterativa não é analisada por recorrência, mas o raciocínio é direto: basta contar quantas vezes os laços aninhados executam.

- O laço **externo** executa $n-1$ passagens (uma para cada elemento a posicionar, exceto o último);
- Na passagem i, o laço **interno** faz comparações de pares adjacentes, cuja quantidade **diminui** a cada passagem;
- Somando todas as iterações do laço interno:

Analisando a versão iterativa

A versão iterativa não é analisada por recorrência, mas o raciocínio é direto: basta contar quantas vezes os laços aninhados executam.

- O laço **externo** executa $n-1$ passagens (uma para cada elemento a posicionar, exceto o último);
- Na passagem i, o laço **interno** faz comparações de pares adjacentes, cuja quantidade **diminui** a cada passagem;
- Somando todas as iterações do laço interno:

Tempo total

$$f(n) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} \Rightarrow T(n) = \Theta(n^2)$$

Mesma complexidade de tempo da versão recursiva, mas sem o custo de **espaço** das chamadas na pilha de recursão.

	Recursiva	**Iterativa**
Tempo (pior/médio caso)	$\Theta(n^2)$	$\Theta(n^2)$
Espaço extra	$\Theta(n)$ (pilha)	$\Theta(1)$
Análise	recorrência $\rightarrow$ árvore	laços aninhados

- As duas versões fazem o **mesmo número de comparações e trocas**;
- A diferença está no uso de memória: cada chamada recursiva ocupa espaço na pilha, enquanto a iterativa reutiliza as mesmas variáveis.

- O Bubble Sort, a cada passagem, compara **pares adjacentes** e troca-os quando fora de ordem, "borbulhando" o maior elemento até o final do trecho.
- **Versão recursiva**: $T(n) = T(n-1) + n$, resolvida pela árvore de recursão (altura $h = n$, custos $n, n-1, \ldots, 1$).
- **Versão iterativa**: dois laços aninhados, somando $(n-1) + (n-2) + \cdots + 1$ comparações.
- Ambas as versões chegam à mesma complexidade final: $T(n) = \Theta(n^2)$ no pior e no caso médio.
- A versão iterativa evita o custo de espaço extra da pilha de recursão.

Dúvidas?

Perguntas? Vamos programar?

